

Reverse Engineering Beleg

Name: Andreas Gröber

MatrNr: 47708

Aufgaben:

- 1. Für gegebenes Binary foo Passphrase ermitteln
- 2. C-Quellcode angeben der die Funktion von foo enthält
- 3. Dokumentation

1. Dokumentation

1.1 DGB-Ausgabe

- disas main im gdb (von foo || Ubuntu)

```
0x080496d5 <+0>: lea    0x4(%esp),%ecx
0x080496d9 <+4>: and    $0xffffffff0,%esp
0x080496dc <+7>: pushl  -0x4(%ecx)
0x080496df <+10>: push  %ebp
0x080496e0 <+11>: mov    %esp,%ebp
0x080496e2 <+13>: push  %ebx
0x080496e3 <+14>: push  %ecx
0x080496e4 <+15>: sub    $0x20,%esp
0x080496e7 <+18>: call   0x80495b0 <__x86.get_pc_thunk.bx>
0x080496ec <+23>: add    $0xa2908,%ebx
0x080496f2 <+29>: movl   $0x0,-0x17(%ebp)
0x080496f9 <+36>: movl   $0x0,-0x13(%ebp)
0x08049700 <+43>: movl   $0x0,-0x10(%ebp)
0x08049707 <+50>: movl   $0x75642403,-0x22(%ebp)
0x0804970e <+57>: movl   $0x6b727072,-0x1e(%ebp)
0x08049715 <+64>: movl   $0x446f726b,-0x1b(%ebp)
0x0804971c <+71>: sub    $0x4,%esp
0x0804971f <+74>: push   $0xa
0x08049721 <+76>: lea    -0x34fec(%ebx),%eax
0x08049727 <+82>: push   %eax
0x08049728 <+83>: push   $0x1
0x0804972a <+85>: call   0x806d060 <write>
0x0804972f <+90>: add    $0x10,%esp
0x08049732 <+93>: sub    $0x4,%esp
0x08049735 <+96>: push   $0xa
0x08049737 <+98>: lea    -0x17(%ebp),%eax
0x0804973a <+101>: push   %eax
0x0804973b <+102>: push   $0x0
0x0804973d <+104>: call   0x806cfa0 <read>
0x08049742 <+109>: add    $0x10,%esp
0x08049745 <+112>: movl   $0x0,-0xc(%ebp)
0x0804974c <+119>: jmp    0x8049794 <main+191>
```

```

0x0804974e <+121>: lea    -0x17(%ebp),%edx
0x08049751 <+124>: mov    -0xc(%ebp),%eax
0x08049754 <+127>: add    %edx,%eax
0x08049756 <+129>: movzbl (%eax),%eax
0x08049759 <+132>: movsbl %al,%edx
0x0804975c <+135>: mov    $0xa,%eax
0x08049761 <+140>: sub    -0xc(%ebp),%eax
0x08049764 <+143>: movzbl -0x22(%ebp,%eax,1),%eax
0x08049769 <+148>: movsbl %al,%eax
0x0804976c <+151>: sub    $0x3,%eax
0x0804976f <+154>: cmp    %eax,%edx
0x08049771 <+156>: je     0x8049790 <main+187>
0x08049773 <+158>: sub    $0x4,%esp
0x08049776 <+161>: push   $0x10
0x08049778 <+163>: lea    -0x34fe1(%ebx),%eax
0x0804977e <+169>: push   %eax
0x0804977f <+170>: push   $0x1
0x08049781 <+172>: call   0x806d060 <write>
0x08049786 <+177>: add    $0x10,%esp
0x08049789 <+180>: mov    $0x1,%eax
0x0804978e <+185>: jmp    0x80497b5 <main+224>
0x08049790 <+187>: addl   $0x1,-0xc(%ebp)
0x08049794 <+191>: cmpl   $0x9,-0xc(%ebp)
0x08049798 <+195>: jle    0x804974e <main+121>
0x0804979a <+197>: sub    $0x4,%esp
0x0804979d <+200>: push   $0x1a
0x0804979f <+202>: lea    -0x34fd0(%ebx),%eax
0x080497a5 <+208>: push   %eax
0x080497a6 <+209>: push   $0x1
0x080497a8 <+211>: call   0x806d060 <write>
0x080497ad <+216>: add    $0x10,%esp
0x080497b0 <+219>: mov    $0x0,%eax
0x080497b5 <+224>: lea    -0x8(%ebp),%esp
0x080497b8 <+227>: pop    %ecx
0x080497b9 <+228>: pop    %ebx
0x080497ba <+229>: pop    %ebp
0x080497bb <+230>: lea    -0x4(%ecx),%esp
0x080497be <+233>: ret

```

1.2 Grober Programmablauf

- Initialisierungen (+variablen+speicher+stack) bis <+64>
- Systemaufruf "Write" <+71 bis +85>
- Systemaufruf "Read" <+85 bis +104>
- bedingter Sprung <+119> und <+191> + <+195>
 - Wenn `-0xc(%ebp) <= 9` Springe zu <+121>
 - Sonst fahre fort
- Fall 1: Fortfahren:
 - Systemaufruf "Write" <+211> sonst passiert nichts besonderes
 - Programm wird Beendet
- Fall 2: Sprung zu <+121>
 - einige Berechnungen bis <+154> dort wird erneut bedingt gesprungen
 - Wenn `%eax` und `%edx` (<+154>) gleich sind Springe auf <+187> dies führt zu einer Schleifen Struktur da von <+191> wieder auf <+121> gesprungen werden kann
 - Sonst wird ein Write gecallt und ans Programm-Ende gesprungen

1.3 Pseudo-Programm

- Mithilfe der gesammelten Informationen kann man eine Art Pseudo-C-Programm erstellen

```
#includes

Init vars (zumindest char* Key vllt noch mehr);

write(write(1, "\n Enter password:\n ",xx);
read(0,Eingabe, yy); //geht aus ausführen Programm hervor

for(i=0;i<=9;noch unbekannt) //geht aus <+191> und <+195> hervor
{
    Key[i] "+ - * /" irgendwas wird zur KeyValue
    oder
    Eingabe[i] "+ - * /" irgendwas wird zum Key

    if(KeyValue[i]!=Eingabe[i]) //falls Key nicht Eingabe dann Falsch
    {
        // Key wird noch zur KeyValue verrechnet
        write(1,"nicht Richtig,xx")
        return 0;
    }
}

write(1, "Richtig",xx); //geht aus ausführung hervor
return 0;
```

- Daraus erkennt man, dass das Passwort höchstwahrscheinlich 10 Zeichen lang ist

1.4 Ermittlung der Passphrase

- Wir brauchen:
 - 1. Key der am Anfang Initialisiert wird
 - 2. die Beziehung bzw. Berechnung in der Key zu Eingabe steht

1.

```
0x080496f2 <+29>: movl    $0x0,-0x17(%ebp)
0x080496f9 <+36>: movl    $0x0,-0x13(%ebp)
0x08049700 <+43>: movl    $0x0,-0x10(%ebp)
0x08049707 <+50>: movl    $0x75642403,-0x22(%ebp)
0x0804970e <+57>: movl    $0x6b727072,-0x1e(%ebp)
0x08049715 <+64>: movl    $0x446f726b,-0x1b(%ebp)
```

Stellt 12 Byte Speicher bereit die dann in 3 Schritten mit jeweils 4 Byte gefüllt werden daraus ergibt sich die folge
Das Ergibt:

```
Keypart_1="0x75 0x64 0x24 0x03" "ud 0x24=$ 0x03"
Keypart_2="0x6B 0x72 0x70 0x72" "krpr"
Keypart_3="0x44 0x6F 0x72 0x6B" "Dork"
```

0x03 ==End of Text

- die Keyparts werden von 3 nach 1 eingelesen und ergeben: Dorkkrprud\$0x03 damit sollte der Key etwa "Dorkkrprud\$" lauten

2.

```
0x0804974e <+121>: lea     -0x17(%ebp),%edx
0x08049751 <+124>: mov     -0xc(%ebp),%eax
0x08049754 <+127>: add     %edx,%eax
0x08049756 <+129>: movzbl (%eax),%eax
0x08049759 <+132>: movsbl %al,%edx
0x0804975c <+135>: mov     $0xa,%eax
0x08049761 <+140>: sub     -0xc(%ebp),%eax
0x08049764 <+143>: movzbl -0x22(%ebp,%eax,1),%eax
0x08049769 <+148>: movsbl %al,%eax
0x0804976c <+151>: sub     $0x3,%eax
0x0804976f <+154>: cmp     %eax,%edx
```

Hier wird der "read"- oder Eingabe-Wert (an stelle i) nach edx geladen, der Schlüsselwert (den wir vorher extrahiert haben) nach eax.

Ich verstehe nicht alles, jedoch wird vor der cmp operation <+154> eine Subtraktion vom Wert im Schlüssel gemacht (um genau zu sein Key[i]-3) <+151>

Das heißt, die Bedingung der If-Anweisung in der Schleife ist etwa wie folgt:
`if(Eingabewert[i]==Key[i]-3)`

Und die Passphrase lautet:"Alohomora!"

2. Programm und Passphrase

2.1 Passphrase

- "Alohomora! führt zum Erfolg"
- sowie "Alohmora!x" ("x" = eine beliebig andere zeichenreihenfolge)
 - damit könnte ich z.B mit "Alohmora!echo fun", noch shellcode ausführen
 - hier jetzt wenig sinnvoll aber eigentümlich für eine Passwortabfrage → Success + das Echo wird ausgeführt

2.2 C-Programm:

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    int i;
    char buffer[10] = {0};
    char *key = "Dorkrprud$"; // geht aus 3 teilen hervor

    write(1, "\n Enter password:\n ",18);
    read(0, buffer, 10);

    // Zeichenweiße abgleichen
    for (i = 0; i <= 9; i++)
    {
        if (buffer[i] != key[i]-3)
        {
            write(1, "\n Incorrect password\n", 20);
            return 0;
        }
    }

    write(1, "Success. Congratulations!\n", 25);
    return 0;
}
```